Reading guide

"The learning problem" (`sec:pf:game`)

Companion to `Chapters/2.Introduction.tex`

This guide walks through the subsection *The learning problem* of the Problem Formulation, one piece at a time. It adds no new model. Everything here points back to an object the thesis already defines. Section 5 spends the most time on the transition kernel P, because the thesis only names it and lists where its parts come from.

Contents

1 What the section is for **1**

2 One timestep in plain words **2**

3 The tuple, element by element **2**
3.1 Spaces versus realisations 3

4 Three games, one difference **3**

5 The transition kernel P **3**
5.1 What a transition kernel is 3
5.2 What is inside P in this thesis 4
5.3 Why "stochastic only via order arrivals" matters 4
5.4 Where the storage layer sits 5

6 Actions and masking **5**

7 The objective $J(\theta)$ **5**

8 Messages belong to the policy **6**

9 Where the congestion feature fits **6**

10 Points worth checking in the thesis text **7**

11 Summary in five lines **7**

1 What the section is for

Up to this point the chapter has defined the warehouse piece by piece: the graph, the agents and how they move, the storage configuration, the tasks, the controllers and what each one can observe. The learning problem section does one thing. It shows that all of these pieces together form a *partially observable stochastic game* (POSG), the standard framework for several agents that act at the same time, each seeing the part of the world its controller's window allows. All three controllers are learned, and Section 4 shows that they play three versions of one game that differ only in what each agent observes.

Key idea. Nothing in the POSG is new. It is a repackaging of objects that are already defined, so that the standard reinforcement-learning machinery (policies, returns, a training objective) can be applied to them.

Why bother? Because once the problem is written as a POSG, the training objective $J(\theta)$ at the end of the section has a precise meaning, and the methods chapter can use off-the-shelf multi-agent RL ideas without redefining the environment.

2 One timestep in plain words

The opening paragraph of the section describes a loop. Here it is with the symbols attached.

1. The warehouse is in some state s_t. *(thesis: "realised state")*
2. Each agent a_i receives its own observation $O_i(s_t)$. Under the section-based and decentralised controllers this is only part of s_t. Under the centralised controller it is all of it.
3. Every agent picks an action $u_i(t)$ at the same time. Together they form the joint action $u_t = (u_1(t), \ldots, u_m(t))$.
4. The warehouse moves to a new state s_{t+1}, drawn from the transition kernel, $s_{t+1} \sim P(\cdot \mid s_t, u_t)$.
5. Each agent receives a reward R_i that scores the step.
6. Repeat until the horizon T.

Two words in the name describe this loop. *Partially observable* is step 2, the agents do not see all of s_t, except under the centralised controller. *Stochastic game* is steps 3 and 4, several agents act together and the next state is random.

3 The tuple, element by element

The thesis writes the game as

$$\mathcal{M} = \big(\mathcal{A},\, \mathcal{S},\, \{\mathcal{U}_i\},\, P,\, \{\mathcal{O}_i\},\, \{O_i\},\, \{R_i\},\, \gamma\big).$$

Table `tab:posg` in the thesis maps each element to its earlier definition. The table below adds a plain meaning for each.

Element	Plain meaning	Where it comes from
$\mathcal{A}$	who acts	the fleet $a_1, \ldots, a_m$
$\mathcal{S}$	everything that could be true of the warehouse at one instant	agent locations $\ell_i(t)$, task sets $\mathcal{Q}_t, \mathcal{B}_t, \mathcal{C}_t$, storage x_t, and the conflict-resolution priority order drawn at episode start
$\mathcal{U}_i$	every action agent i could ever take	union of the per-vertex sets $\mathcal{U}(v)$, see Section 6
P	the rules of the warehouse, how one state leads to the next	movement, conflicts, task lifecycle, assignment, storage update, order arrivals, see Section 5
$\mathcal{O}_i$	every observation agent i could receive	the range of O_i, one per architecture
O_i	how agent i's observation is cut out of the state	`eq:obsfunction`, see Section 4
R_i	the score each step earns	deliberately left open, set in the methods chapter
γ	how much the future counts	a training parameter

3.1 Spaces versus realisations

The section insists on one distinction, and it is worth getting straight before reading further.

- A *space* or *function* is the set of all possibilities or the rule. Examples are $\mathcal{S}$, $\mathcal{O}_i$, O_i, P, R_i.
- A *realisation* is one concrete value that actually occurs in a run. Examples are s_9, $o_i(t)$, a reward of -1 at $t = 9$.

The task example, where a_2 is assigned τ_1, makes it concrete. s_9 is one realised state at $t = 9$. It records where every agent is (including a_2, just assigned τ_1), that $\tau_1 \in \mathcal{B}_9$, and the storage configuration x_9. $\mathcal{S}$ is the space all such snapshots live in. The letter clash between $\mathcal{O}_i$ (space) and O_i (function) is exactly why the thesis points to the notation appendix.

4 Three games, one difference

All three controllers are learned policies from one family, trained the same way. What separates them is only what each agent gets to see. The thesis says this with one equation, `eq:obsfunction`,

$$O_i(s_t) = \begin{cases} (s_t,\, i) & \text{centralised,} \\ (s_t^{(q)},\, i) \text{ with } \ell_i(t) \in Y_q & \text{section-based,} \\ o_i(t) & \text{decentralised.} \end{cases}$$

Key idea. Everything else in $\mathcal{M}$ is identical across the three. Same agents, same states, same actions, same P, same rewards, same γ. So the three learned controllers play three versions of one game that differ only in O_i, and that is exactly the contrast RQ2 asks about.

Read the three cases in plain words.

- **Centralised.** Agent a_i's head sees the whole state plus its own index i, so it knows which agent it is choosing for. This game is fully observed, which is a special case a POSG already covers, not a different framework.
- **Section-based.** Agent a_i's head sees the zone it is in, plus the halo the boundary protocol passes across. When the agent crosses into another zone, its observation switches with it.
- **Decentralised.** Agent a_i sees only its own $o_i(t)$, the three pieces of the observation subsection (field of view, direction to target, crowding).

The windows are nested. The decentralised observation can be computed from the section-based one, since the halo has the same depth as the field of view, and the section-based one from the centralised one. Messages do not spoil this, because they are not part of any observation. They belong to the decentralised policy, see Section 8. The equation says what a controller *may* condition on. Whether the realised network uses all of it is a methods question, still open for the centralised network (the pooling point in `sec:method:model`).

5 The transition kernel P

5.1 What a transition kernel is

A transition kernel answers one question.

> *If the warehouse is in state s and the agents take joint action u, how likely is each possible next state s'?*

Formally, $P(s' \mid s, u)$ is a probability distribution over $\mathcal{S}$ for every pair (s, u), so the probabilities over all possible s' sum to one. Writing $s_{t+1} \sim P(\cdot \mid s_t, u_t)$ means "the next state is drawn from that distribution". The thesis uses exactly this form in the coupling section,

$$s_{t+1} \sim P(\cdot \mid s_t, u_t), \qquad u_t \sim \pi(\cdot \mid s_t).$$

Key idea. P is the environment. The policy chooses u_t. Everything that happens *after* the choice, until the next state, is P.

5.2 What is inside P in this thesis

The thesis never writes P as one formula. Table `tab:posg` instead lists what P is made of, `eq:transition` with conflict resolution, the shared assignment rule, and `eq:storageupdate`, and says it is "stochastic only via order arrivals". The thesis also says why the two shared rules sit in P. Neither is an action of any agent, and both are the same for every controller, so they belong to the environment, not to a policy. Unpacking that, one step of P does the following, all of it already defined earlier in the chapter.

(a) **Move the agents.** Each agent either waits or moves along one edge, `eq:transition`, $\ell_i(t+1) = \ell_i(t)$ or $(\ell_i(t), \ell_i(t+1)) \in E$.

(b) **Resolve conflicts.** If two agents would end on the same vertex (`eq:vertexconflict`) or swap across one edge (`eq:swapconflict`), the conflict-resolution mechanism decides who yields. It works through the agents in a priority order that is drawn once, at the start of the episode, and then kept. That order is part of the state, so the mechanism is deterministic given the state and adds no randomness.

(c) **Update the tasks.** A task moves from $\mathcal{B}_t$ to $\mathcal{C}_{t+1}$ when its agent delivers it. The target $q_i(t)$ switches from pickup to delivery at pickup.

(d) **Update storage, but only on the slow clock.** At epochs $t \in \{\Delta, 2\Delta, \dots\}$ the storage rule F produces a new x_t (`eq:storageupdate`). Between epochs x_t stays fixed.

(e) **Add new orders.** New orders are drawn from P_{ord} and turned into tasks in $\mathcal{Q}_{t+1}$. *This is the only random step.*

(f) **Assign tasks.** Free agents are matched to waiting tasks by the one assignment rule every controller shares. No policy chooses this, so it is part of the environment. Ties are broken from the state alone. Tasks released at the same time go in task order, and equally close agents are ranked by the same priority order the conflict-resolution step uses.

The methods chapter runs orders and assignment at the *start* of a timestep, before routing, and resolution and storage after. Read as a POSG, nothing changes. The end of one timestep and the start of the next together make up one application of P.

5.3 Why "stochastic only via order arrivals" matters

Steps (a) to (d) and (f) are deterministic. The modelling assumption "deterministic execution" says an accepted action always succeeds, so a robot told to move does move. The only thing the agents cannot predict is which orders arrive next. One helpful way to read P is therefore in two parts,

$$s_{t+1} = f\big(s_t,\, u_t,\, z_{t+1}\big), \qquad z_{t+1} \sim P_{\text{ord}},$$

where f collects steps (a) to (d), turning the new orders z_{t+1} into tasks, and step (f). This split is a reading aid, not notation the thesis uses. It shows that if you fixed the order stream, the whole run would be completely determined by the agents' actions.

Small example. Take the running example at $t = 9$. Agent a_2 has just been assigned τ_1 and heads for pickup vertex A. Suppose a_2 chooses to move one vertex towards A and no other agent wants that vertex. Then steps (a) to (c) fix a_2's next location exactly. Unless $t = 9$ is a storage epoch, $x_{10} = x_9$. The only uncertainty in s_{10} is which new tasks appear in $\mathcal{Q}_{10}$. If no orders could arrive, $P(\cdot \mid s_9, u_9)$ would put probability one on a single next state.

5.4 Where the storage layer sits

It can seem odd that the storage rule F is inside the environment, since storage is one of the two layers the thesis studies. The reading is that, from the routing policy's point of view, F is a fixed rule that runs on its own clock. The routing agents do not choose storage. They experience its effect as part of how the world evolves, through pickup vertices moving. That is the coupling the next section of the thesis (*How the two layers close the loop*) builds on.

6 Actions and masking

A POSG normally has a fixed action set per agent. In a graph, the legal actions depend on where the agent stands. At vertex v only $\mathcal{U}(v) = \{\text{wait}\} \cup \{\text{move}(w) : w \in N^+(v)\}$ is legal. The thesis resolves this in the usual way.

- Give every agent the big set $\mathcal{U}_i = \bigcup_v \mathcal{U}(v)$, all moves to all vertices, plus wait.
- At each state, switch off (mask) everything outside $\mathcal{U}(\ell_i(t))$.

Example. At a vertex with one outgoing edge to w, the agent really has two choices, wait or move(w). All other entries of $\mathcal{U}_i$ are masked.
The masking happens in the policy, not in P. The masked actions get probability zero because they are removed before the policy samples (`eq:mask` in the methods chapter). P only ever assigns probabilities to next states.

7 The objective $J(\theta)$

$$\max_\theta J(\theta) = \max_\theta \mathbb{E}_{\pi_\theta}\Big[\sum_{t=0}^{T-1} \gamma^t \, \frac{1}{m} \sum_{i=1}^{m} R_i(t)\Big]$$

Read it from the inside out.

- $\frac{1}{m}\sum_i R_i(t)$ is the fleet's average reward at step t. Averaging, not summing per agent, is what makes this a *team* objective. The agents are cooperative and share one set of weights.
- γ^t with $\gamma \in [0, 1)$ makes later reward count less. With $\gamma = 0.99$, reward 100 steps ahead counts about $0.99^{100} \approx 0.37$ as much as reward now.
- $\mathbb{E}_{\pi_\theta}$ averages over everything random in a run, the policy's own sampling of actions and the order arrivals inside P.
- $\max_\theta$ says training searches for the weights that make this expected team return as large as possible.

$R_i(t)$ is shorthand for $R_i(s_t, u_t, s_{t+1})$, the reward for agent i on the step from s_t to s_{t+1} under joint action u_t.
The same objective covers all three learned controllers. Only what θ parameterises changes, the shared per-agent network in the decentralised case, or the global or zone-level network otherwise. A classical reference planner is reported in the results as a baseline, but it is not part of this problem and is not trained.

One objective, different credit. If all three controllers maximise the same team average, why would they differ in how much congestion they cause? Because the objective is the *target*, and how each update credits reward to an agent's own action is a separate training choice. An update can use the mean reward over the agents a controller moves, or each agent's own R_i. With its own R_i, an agent is not charged for the delay it causes others, so it has no reason to avoid it. That choice is fixed in the methods chapter (it is still marked open there), and it is what makes the predicted ordering, centralised least exposed to congestion and decentralised most, consistent with one shared objective.

The critic sees what the policy sees. PPO trains a critic next to each policy, a value estimate used only while learning. In this thesis each critic sees exactly the window of its own policy, so a decentralised learner also *learns* from local information only. That is on purpose. It means the architectures differ in training as well as in acting, and a decentralised learner is never taught the cost of congestion it causes out of its sight, which is the effect the comparison measures.

Why R_i is left open. The reward is a tool for getting a good policy, not the thing the thesis finally measures. The final measures are the system-level criteria (throughput, service time and so on). The reward can therefore be redesigned without changing the problem, which is why it is fixed in the methods chapter instead.

8 Messages belong to the policy

A decentralised agent can talk to the agents within its field of view. What it sends is not chosen by hand. The agent's own network computes a message from its own observation, and training decides what is worth sending. This is how CRAMP and PICO do it, and the thesis follows them.

Key idea. A learned message depends on the network's weights, not only on the state. So it cannot be part of the observation, which is a fixed rule applied to the state. It is part of the policy instead, a communication step inside the decentralised controller.

Three consequences follow.

- $o_i(t)$ has three pieces, field of view, direction to target and crowding. Messages are not a fourth piece.
- The observation functions stay nested exactly. Nothing a decentralised agent observes is hidden from the other two controllers.
- Communication has its own on/off switch, the conditional RQ5. With it off, the agent acts from $o_i(t)$ alone.

Messages travel only one hop, from agents within the field of view. More hops would pass on what agents further away send, which would let a decentralised agent learn about agents it cannot see and widen its window.

9 Where the congestion feature fits

$\delta_i(t)$ can be read from every controller's window, and for the decentralised controller it is one of the three pieces of $o_i(t)$. So the policy can see it, and a reward designer may use it. Whether the policy is allowed to react to it is the experimental toggle "congestion sensitivity: disabled/enabled". That toggle is a choice of the experiment, not a property of any architecture.

10 Points worth checking in the thesis text

An earlier reading of the section for this guide turned up four places where the wording was looser than the rest of the formulation. Three are now fixed in the thesis (tag `[A12]`). The same pass found two more, and all of them are now fixed.

Fixed.

1. **Masked actions and P.** The text said the masked actions "have zero probability under P", but P assigns probabilities to next states, not to actions. It now says they get zero probability under the policy, since they are masked out before sampling.

2. **The signature of R_i.** The thesis now states once that $R_i(t)$ is shorthand for $R_i(s_t, u_t, s_{t+1})$.

3. **O_i and messages.** The worry was that messages come from other agents' policies, so O_i might not be a function of s_t alone. It was first settled by fixing the message to two numbers read from the state. It is now settled the other way (tag `[A14]`). Messages are learned, so they are not part of the observation at all but part of the decentralised policy, and O_i is a function of the state by construction. See Section 8.

4. **Ties in the assignment rule.** Once assignment is part of P, P needs it to be a function, and ties are common when every edge costs the same. Tasks released at the same time now go in task order, and equally close agents are ranked by the conflict-resolution priority order. That order is already in the state and is redrawn each episode, so no agent is favoured. Using communication to settle ties was ruled out, because assignment must be the same for every controller and only the decentralised one communicates.

5. **Which critic each controller uses** (tag `[A13]`). The thesis used to define centralised training with decentralised execution without saying whether it used it. It now says each controller's critic sees exactly the window its own policy sees, and gives the reason. The window should limit how a controller learns as well as how it acts, and a local critic keeps the congestion contrast measurable.

6. **Two running examples.** The section said "continuing the running example" right after the observation subsection, which has its own running example with agent a_1, so the phrase pointed at the wrong one. It now names the task example of the task-route figure, where a_2 is assigned τ_1 at $t = 9$.

11 Summary in five lines

1. The warehouse problem is a POSG, many agents, one shared environment, and three observation functions, one per controller.

2. Every element of the POSG was already defined earlier in the chapter, and only O_i differs between the three learned controllers.

3. P is the environment's rulebook, movement, conflict resolution, task updates, assignment, slow storage updates and order arrivals.

4. Only the order arrivals are random, everything else in P is deterministic.

5. Training maximises the discounted team average reward, and how each update credits that reward is a separate choice made in the methods chapter.